

## ORM-модели для системы блога

Необходимо разработать бэкенд для простого блога. Нужно спроектировать базу данных и реализовать слой доступа к данным с использованием SQLAlchemy ORM и PostgreSQL

В рамках реализации данного модуля сформулированы следующие функциональные требования и ограничения:

1. Необходимо использовать SQLAlchemy фреймворк и методологию ORM (declarative style) и драйвер psycopg2. Запрещено писать «сырые» SQL-запросы, использовать только подходы ORM.
2. В качестве СУБД использовать PostgreSQL. Для удобства демонстрации результатов в рамках данной работы предложено использовать pgAdmin.
3. Код должен быть разбит на логические файлы: .env, database.py, models.py, crud.py, main.py.

### Настройка среды

Для выполнения задания необходимо выполнить следующие подготовительные этапы.

1. Настроить PostgreSQL и pgAdmin. В рамках работы это предлагается выполнить средствами контейнеров Docker.
2. Настроить виртуальную среду с необходимыми библиотеками.
3. Настроить файл .env.

#### Этап 1. Настройка СУБД PostgreSQL и pgAdmin

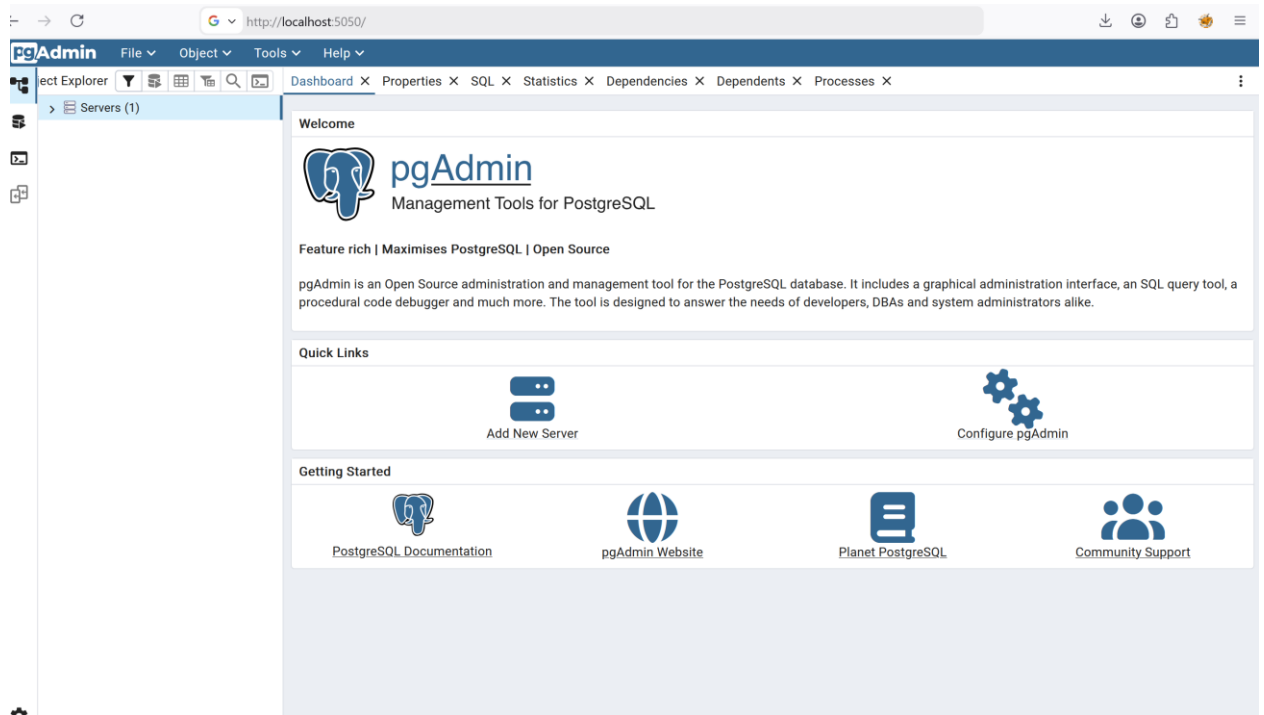
1. Установите Docker. Как установить Docker и что это такое, смотрите в документации. Ссылки:
  - a. Для установки на Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>
  - b. Для установки на Linux: <https://docs.docker.com/engine/install/>
  - c. Для установки на MacOS <https://docs.docker.com/desktop/setup/install/mac-install/>
2. Далее необходимо создать папку своего проекта и в нее получить файлы репозитория по ссылке: <https://github.com/khezen/compose-postgres>
3. Далее в терминале перейти в папку с файлами репозитория и выполнить команду `docker compose up`. Далее у вас должны создаться и запуститься контейнеры. Пример на рисунке 1.

```
victoria_s@victorialinux: /opt/compose_files/postgres_pgadmin/compose-postgres$ sudo docker compose up
[+] up 39/39
✔Image dpape/pgadmin4          Pulled
✔Image postgres                Pulled
✔Network compose-postgres_postgres Created
✔Volume compose-postgres_pgadmin Created
✔Volume compose-postgres_postgres Created
✔Container postgresdb         Created
✔Container pgadmin             Created
```

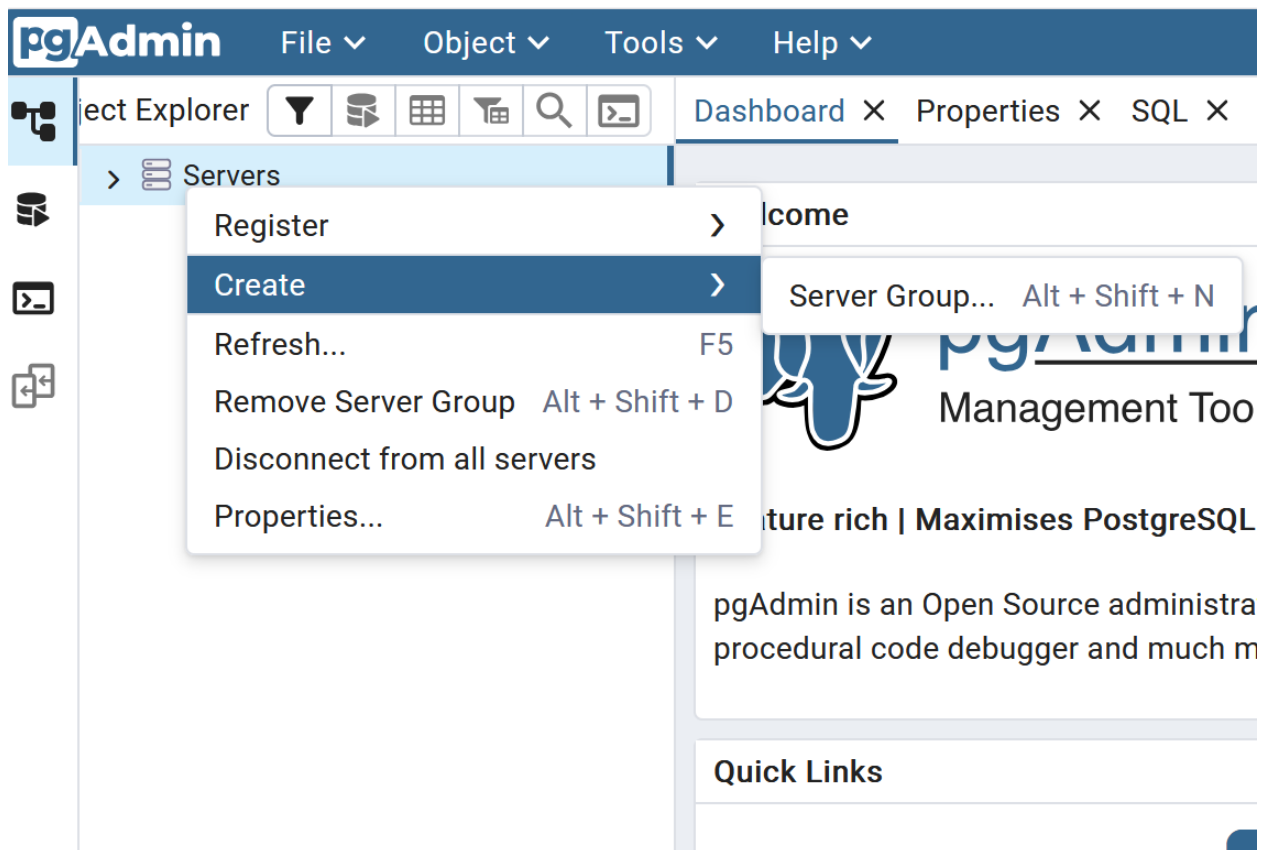
Рис. 1. Пример ввода команды в терминал и результат

4. Далее проверьте, что у вас запускается pgAdmin. Для этого перейдите по ссылке <http://localhost:5050/>

Если вы все сделали правильно, то у Вас в браузере появится такое окно:



5. Далее необходимо подключиться из pgAdmin к СУБД PostgreSQL. Для этого нажимаете правой кнопкой мыши на Servers -> Create -> Server Group



Откроется следующее окно:

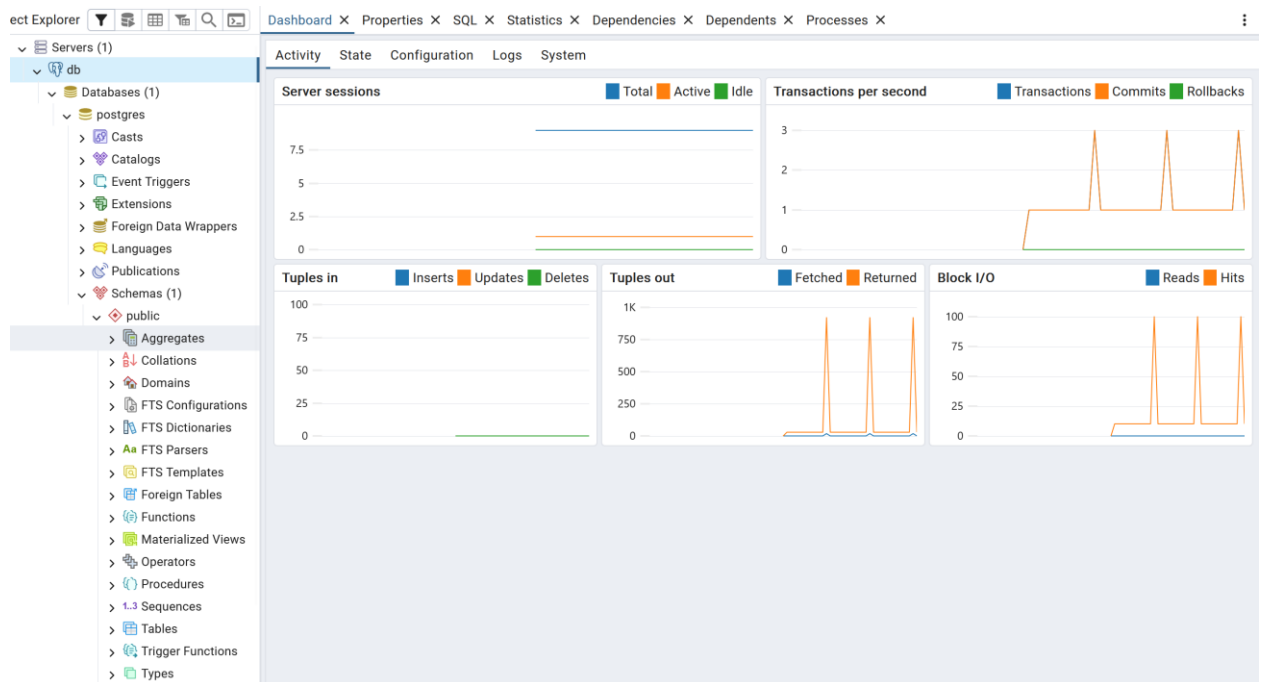
The screenshot shows a configuration window titled 'db' with a close button (X) in the top right corner. The 'General' tab is selected, showing fields for 'Name' (containing 'db'), 'Server group' (a dropdown menu showing 'Servers'), 'Background' (with a close button X), 'Foreground' (with a close button X), and 'Comments' (a large text area). At the bottom, there are icons for help (i) and a question mark (?), and buttons for 'Close', 'Reset', and 'Save'.

6. Далее переходите во вкладку Connection и вводите данные:

The screenshot shows the same 'db' configuration window, but with the 'Connection' tab selected. It contains fields for 'Host name/address' (postgresdb), 'Port' (5432), 'Maintenance database' (postgres), 'Username' (postgres), 'Kerberos authentication?' (a toggle switch), 'Password' (a masked field with dots), 'Save password?' (a toggle switch), and 'Role' (an empty field). A note below the password field states: 'In edit mode the password field is enabled only if Save Password is set to true.' At the bottom, there are icons for help (i) and a question mark (?), and buttons for 'Close', 'Reset', and 'Save'.

Пароль находится в .env файле клонированного ранее репозитория с compose файлом.

Далее нажимаете save и у вас должно отобразиться следующее окно:



Если Вы дошли до этого этапа, то рабочая среда для работы postgresSQL готова.

## Этап 2. Настройка виртуальной среды.

На этом этапе необходимо создать новую виртуальную среду python и выполнить установку пакетов:

```
(env)>>pip install sqlalchemy psycopg2 python-dotenv
```

## Этап 3. Настройка переменных окружения (настройка .env).

В корневой папке проекта создайте файл с названием .env. Внутри этого файла пропишите параметры для подключения к БД.

```
# .env
# Настройки подключения к PostgreSQL
# Этот файл НЕ должен попадать в систему контроля версий (git)!
```

```
DB_USER=postgres
DB_PASSWORD=postgres
DB_PORT=5432
DB_HOST=localhost
DB_NAME=postgres
```

Сохраните файл. Среда для создания приложения готова, Можно приступать к разработке.

## Краткая теоретическая справка

ORM (Object-Relational Mapping) — архитектурный подход и набор библиотек, позволяющих работать с реляционными базами данных через объектно-ориентированный интерфейс. Вместо написания SQL-запросов программист оперирует классами и объектами, а ORM автоматически транслирует их в SQL, управляет соединениями, транзакциями и преобразованием типов.

В рамках ORM парадигмы можно выстроить следующие соответствия:

Класс из кода преобразуется в отдельную таблицу, причем атрибуты класса являются колонками. Объект или экземпляр класса представляет собой строку таблицы (запись). Свойство объекта – это значение ячейки, а ссылка на другой объект – это внешний ключ.

Основные преимущества и недостатки ORM-парадигмы представлены в таблице:

| Достоинства                                                                                                                 | Недостатки                                             |
|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Код пишется на языке программирования (например, Python), нет необходимости в написании SQL-запросов, ускоряется разработка | сложные запросы могут быть неоптимальны (N+1 проблема) |
| Защита от SQL-инъекций                                                                                                      | Трудности с разработкой сложных запросов               |
| Легко сменить СУБД (SQLite → PostgreSQL)                                                                                    | Иногда неочевидное поведение с кэшем или сессией       |
| Упрощается поддержка кода                                                                                                   |                                                        |

SQLAlchemy — самый популярный ORM для Python. Его особенность в том, что это не монолит: он состоит из двух независимых слоёв.

SQLAlchemy обладает уникальной двухуровневой архитектурой: Core + ORM:

- SQLAlchemy Core (Низкий уровень), который работает через Table, select, insert. Данный слой генерирует SQL, но возвращает словари или строки, а не объекты.
- SQLAlchemy ORM (Высокий уровень), которая является надстройкой над Core. Он работает через Session, Mapper, Declarative Base и возвращает объекты классов.

Более подробно о данной библиотеке можно узнать в документации.

### Создание приложения

1. Для начала создадим файл database.py. В рамках него проверим подключение к нашей созданной ранее БД. Для этого используем следующий код:

```
import os
from dotenv import load_dotenv

# Загружаем переменные из файла .env
load_dotenv()

# Читаем переменные окружения (с значениями по умолчанию)
DB_USER = os.getenv("DB_USER", "postgres")
DB_PASSWORD = os.getenv("DB_PASSWORD", "postgres")
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_PORT = os.getenv("DB_PORT", "5432")
DB_NAME = os.getenv("DB_NAME", "blog_db")

# Формируем строку подключения для SQLAlchemy
DATABASE_URL =
f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
```

Этот код позволяет нам получить параметры для подключения к БД, которые заданы в файлу `.env`. Далее выполняем непосредственное подключение к БД:

```
engine = create_engine(DATABASE_URL, echo=False)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()
```

Этот код настраивает соединение с базой данных и создаёт инструменты для работы с ней через SQLAlchemy (популярная библиотека для работы с БД в Python). Разберём каждую строку:

### 1. `engine = create_engine(DATABASE_URL, echo=False)`

Этот код создаёт движок – ядро подключения к базе данных:

- `DATABASE_URL` – строка подключения, которая включает следующую информацию:

`postgresql://user:password@host/dbname`

Строка подключения в нашем случае сформирована, как f-строка из соответствующих полей и присвоена в переменную `DATABASE_URL`.

- `echo=False` — отключает логирование всех SQL-запросов в консоль (если `True` — будет видно каждый выполненный запрос)

Движок управляет пулом соединений, транзакциями и диалектом конкретной БД

### 2. `SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)`

Создаёт сессии для общения с БД:

- `bind=engine` — привязывает сессию к созданному движку
- `autoflush=False` — не отправляет изменения в БД автоматически (нужно явно вызвать `flush()`)
- `autocommit=False` — отключает автоматический коммит (транзакции управляются вручную через `commit()/rollback()`)

Сессии нужны для выполнения запросов, добавления/изменения объектов.

### 3. `Base = declarative_base()`

Создаёт базовый класс для моделей. Модель – это класс, который будет соответствовать таблице в БД. Особенности класса `Base` заключаются в следующем.

- Все таблицы (в виде Python-классов) будут наследоваться от этого класса.
- SQLAlchemy использует `Base` для метаданных о схемах, отношениях, индексах

На этом пока закончим редактирование файла `database.py` и перейдем к созданию файла `models.py`

2. Создадим отдельный файл `models.py`. Этот файл будет включать в себя набор классов, соответствующих таблицам в базе данных.

В рамках данного проекта будет создано 3 таблицы:

Таблица «Author», которая включает поля:

- id: Integer, primary key
- name: String(100), nullable=False, unique=True
- email: String(150), nullable=False, unique=True
- created\_at: DateTime, default=datetime.utcnow

Таблица «Post», которая включает поля:

- id: Integer, primary key
- title: String(200), nullable=False
- content: Text, nullable=False
- published: Boolean, default=False
- author\_id: Integer, ForeignKey('authors.id')
- created\_at: DateTime, default=datetime.utcnow

Таблица «Comments», которая включает поля:

- id: Integer, primary key
- text: Text, nullable=False
- post\_id: Integer, ForeignKey('posts.id')
- author\_name: String(100), nullable=False # имя комментатора (не ссылка на Author)
- created\_at: DateTime, default=datetime.utcnow

### Краткая информационная справка

Реляционная база данных — это способ хранения информации в виде таблиц, похожих на листы Excel. Каждая таблица представляет одну сущность, например пользователей, товаров или заказов. Строки таблицы — это отдельные записи, а столбцы — это атрибуты, которые описывают каждую запись, такие как имя, цена или дата создания. Слово «реляционная» происходит от английского relation, что в данном контексте означает не «связь» в бытовом смысле, а математическое понятие «отношение» — то есть, по сути, просто таблицу. Однако на практике реляционные базы данных так называются потому, что они позволяют устанавливать отношения между разными таблицами.

#### Ключевые понятия:

Таблица (сущность) — например: «Пользователи».

Строка (кортеж, запись) — например: «Пользователь Иван».

Столбец (атрибут, поле) — например: «Имя», «Возраст».

Простыми словами: «Реляционная» — значит «основанная на отношениях (relations) между таблицами».

#### Первичный ключ

У каждой строки в таблице должен быть уникальный идентификатор, чтобы мы могли точно на неё указать. Этот идентификатор называется первичным ключом (Primary Key). Первичный ключ обладает двумя главными свойствами: он уникален в пределах всей таблицы и никогда не может быть пустым, то есть не может содержать значение NULL.

Чаще всего в роли первичного ключа выступает целочисленное поле с автоматическим увеличением: `id` со значениями 1, 2, 3 и так далее. Поиск по первичному ключу — самый быстрый способ найти нужную запись.

### Внешний ключ

Внешний ключ (Foreign Key) — это столбец в одной таблице, который ссылается на первичный ключ другой таблицы. Именно внешние ключи создают связи между таблицами. Например, в таблице «Посты» может быть столбец «`автор_id`», который хранит `id` пользователя из таблицы «Пользователи». Это и есть внешний ключ. Благодаря внешнему ключу база данных может следить за целостностью данных: вы не сможете добавить пост с `автор_id`, которого не существует в таблице пользователей. Также вы не сможете удалить пользователя, если у него остались посты, если только вы специально не разрешили каскадное удаление.

### Типы связей между таблицами

Существует три основных типа связей между таблицами.

**Связь «один-к-одному»** означает, что одной записи в первой таблице соответствует ровно одна запись во второй таблице, и наоборот. Например, у одного пользователя может быть только один паспорт, и один паспорт принадлежит только одному пользователю. На практике такая связь встречается реже всего.

**Связь «один-ко-многим»** — это самый распространённый тип связи. Одна запись из первой таблицы может быть связана со многими записями из второй таблицы. Например, один пользователь может написать много постов. При этом внешний ключ всегда находится на стороне «многих», то есть в таблице постов будет столбец «`автор_id`», а в таблице пользователей никакого «списка постов» в виде столбца не будет.

**Связь «многие-ко-многим»** возникает, когда одна запись из первой таблицы может быть связана со многими записями из второй, и наоборот. Классический пример — студенты и курсы: один студент может учиться на многих курсах, и в каждом курсе может учиться много студентов. В реляционных базах данных такую связь нельзя выразить напрямую с помощью одного внешнего ключа. Вместо этого создаётся третья, вспомогательная таблица — она называется связующей. В ней хранятся только два внешних ключа: один на `id` студента, другой на `id` курса. Каждая строка в этой таблице означает, что данный студент записан на данный курс.

### Как это связано с ORM

Понимание первичных и внешних ключей, а также типов связей необходимо для работы с любым ORM, включая SQLAlchemy. В ORM каждая таблица превращается в класс Python, первичный ключ — в атрибут с аннотацией `primary_key`, внешний ключ — в атрибут `ForeignKey`, а связи описываются с помощью функции `relationship`. Например, связь «один-ко-многим» в коде будет выглядеть так: у класса `User` появится атрибут `posts`, который ведёт себя как список объектов `Post`, хотя в базе данных никакого списка нет — там есть только внешний ключ `author_id` в таблице `posts`. ORM автоматически превращает работу с такими атрибутами-списками в запросы с `JOIN` и подзапросы.

Тогда, связи в нашем проекте будут следующие:

Связь `Post`: один ко многим с `Author` (один автор → много постов)

Связь `Post`: один ко многим с `Comment` (один пост → много комментариев)

Связь `Comment`: многие к одному с `Post`



В файле models.py импортируем все необходимые объекты и функции:

```
from sqlalchemy import Column, Integer, String, Text, Boolean, DateTime,
ForeignKey
from sqlalchemy.orm import relationship
from datetime import datetime
from database import Base
```

Этот импорт позволяет использовать объекты, характеризующие основные типы колонок таблиц и инструменты для создания связей между ними из SQLAlchemy, которые используются для определения моделей (таблиц) в декларативном стиле.

**Column** – базовый класс для определения колонки в таблице.

**Integer** – целочисленный тип (INT, INTEGER в БД).

**String** – строковый тип фиксированной или переменной длины (VARCHAR в БД).

**Text** – длинный текст без ограничения длины (TEXT в БД).

**Boolean** – логический тип (BOOL/BOOLEAN в БД).

**DateTime** – дата и время (TIMESTAMP/DATETIME в БД).

**ForeignKey** создаёт внешний ключ для связи между таблицами.

Тогда, создадим таблицы текущего проекта.

```
class Author(Base):
    __tablename__ = "authors" # Имя таблицы в БД

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False, unique=True)
    email = Column(String(150), nullable=False, unique=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Связь: один автор → много постов
    # back_populates создаёт обратную связь: post.author даст объект автора
    posts = relationship("Post", back_populates="author", cascade="all, delete-
orphan")

    def __repr__(self):
        return f"<Author(name='{self.name}', email='{self.email}')>"
```

В данном коде мы создали класс для таблицы авторов, задали там первичный ключ и типы данных для всех колонок. `__tablename__` представляет собой имя таблицы в БД. Это имя той таблицы, с которой будет ассоциирован созданный класс.

\Так же построили связь между таблицей авторов и постов. Параметр `back_populates` – это как двусторонняя связь между двумя таблицами.

Функция `__repr__` вспомогательная, она определяет удобочитаемое представление объекта при выводе в консоль или отладке.

Создадим все оставшиеся классы для описанных ранее таблиц:

```
class Post(Base):
    __tablename__ = "posts"
```

```

id = Column(Integer, primary_key=True, index=True)
title = Column(String(200), nullable=False)
content = Column(Text, nullable=False)
published = Column(Boolean, default=False)
created_at = Column(DateTime, default=datetime.utcnow)

# Внешний ключ на таблицу authors
author_id = Column(Integer, ForeignKey("authors.id"), nullable=False)

# Связи с другими таблицами
author = relationship("Author", back_populates="posts")
comments = relationship("Comment", back_populates="post", cascade="all,
delete-orphan")

def __repr__(self):
    return f"<Post(title='{self.title}', published={self.published})>"

class Comment(Base):
    __tablename__ = "comments"

    id = Column(Integer, primary_key=True, index=True)
    text = Column(Text, nullable=False)
    author_name = Column(String(100), nullable=False) # Имя комментатора (не
ссылка на Author)
    created_at = Column(DateTime, default=datetime.utcnow)

    # Внешний ключ на таблицу posts
    post_id = Column(Integer, ForeignKey("posts.id"), nullable=False)

    # Связь с постом
    post = relationship("Post", back_populates="comments")

    def __repr__(self):
        return f"<Comment(post_id={self.post_id}, author='{self.author_name}')>"

```

Для того, чтобы описанные таблицы создались в нашей базе данных postgres, в конце файла models.py введем следующий код:

```

if __name__ == "__main__":
    from database import engine, Base
    Base.metadata.create_all(bind=engine) # Создаст все таблицы, если их нет
    print("✓ Таблицы созданы!")

```

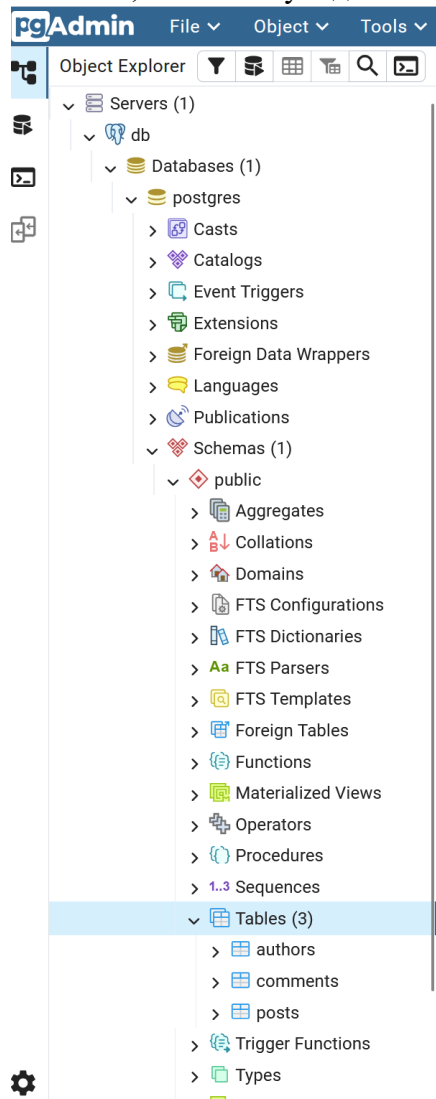
Если до этого этапа вы ввели все верно, то у Вас будет выведено в консоль следующее сообщение:

```

2026-05-05 11:27:18,726 INFO sqlalchemy.engine.Engine COMMIT
✓ Таблицы созданы!

```

Теперь проверим, создались ли таблицы через интерфейс pgAdmin. Если открыть созданную БД в боковой панели слева, то можно увидеть созданные нами таблицы:



Теперь создадим некоторые функции, позволяющие записывать, редактировать и удалять данные в нашей БД.

3. Создадим логику записи и удаления данных в БД на основе шаблона CRUD (Create, Read, Update, Delete).

В папке проекта создадим новый файл `crud.py`

Для начала выполним все необходимые импорты.

```
from sqlalchemy.orm import Session
from models import Author, Post, Comment
from datetime import datetime
```

Первая функция, которую мы создадим – функция по добавлению нового автора в таблицу.

```
def create_author(session: Session, name: str, email: str) -> Author:
    """Создает нового автора, возвращает объект"""
    new_author = Author(name=name, email=email)
    session.add(new_author)
    session.commit()
    session.refresh(new_author) # Обновляем объект, чтобы получить id
    return new_author
```

Данный код реализует функцию для создания новой записи автора в базе данных с использованием ORM SQLAlchemy. Далее рассмотрим построчно написанную функцию:

**new\_author = Author(name=name, email=email)**

Создаётся экземпляр ORM-модели Author. На этом этапе объект существует только в памяти Python, в базе данных его ещё нет.

**session.add(new\_author)**

Объект привязывается к текущей сессии и помечается как pending. SQLAlchemy запомнит, что при следующей синхронизации с БД нужно выполнить INSERT, но сам запрос пока не отправляется.

**session.commit()**

Фиксирует транзакцию. Под капотом SQLAlchemy сначала делает flush() (отправляет INSERT в базу), а затем подтверждает транзакцию. После этой строки запись физически появляется в таблице.

**session.refresh(new\_author)**

Принудительно загружает свежие данные объекта из базы. Это нужно, чтобы подтянуть поля, которые заполняются автоматически на стороне СУБД: например, автоинкрементный id, значения по умолчанию, created\_at, триггеры и т.д. Без refresh() эти атрибуты могут оставаться None или неактуальными.

**return new\_author**

Возвращает полностью инициализированный объект, готовый к использованию в остальном коде.

Далее реализуем функцию поиска автора по его email.

```
def get_author_by_email(session: Session, email: str) -> Author | None:
    """Ищет автора по email"""
    return session.query(Author).filter(Author.email == email).first()
```

Разберем построчно:

**session.query(Author)**

Иницирует построение запроса к таблице, привязанной к ORM-модели Author.

**.filter(Author.email == email)**

Добавляет условие WHERE email = :email. SQLAlchemy автоматически параметризует запрос, защищая от SQL-инъекций.

**.first()**

Выполняет запрос в БД, добавляя LIMIT 1.

Далее создадим функцию создания постов.

```
def create_post(session: Session, title: str, content: str,
                author_id: int, published: bool = False) -> Post:
    """Создает новый пост"""
    new_post = Post(
        title=title,
        content=content,
        author_id=author_id,
        published=published
    )
    session.add(new_post)
    session.commit()
    session.refresh(new_post)
    return new_post
```

Данная функция по структуре идентична ранее описанной функции создания авторов.

Создадим функцию отбора всех опубликованных постов. То есть это все посты, которые имеют статус «Опубликовано». В таблице для этого существует поле published и нас интересуют все посты, у которых это поле имеет значение true.

```
def get_published_posts(session: Session, limit: int = 10) -> list[Post]:
    """Возвращает только опубликованные посты (published=True)"""
    return session.query(Post).filter(Post.published == True).limit(limit).all()
```

В данной функции задается количество авторов, которое будет выведено при исполнении запроса (параметр Limit).

Далее напишем функцию, которая выводит посты, опубликованные конкретным автором.

```
def get_posts_by_author(session: Session, author_id: int, limit: int = 10) -> list[Post]:
    """Возвращает все посты конкретного автора"""
    return session.query(Post).filter(Post.author_id == author_id).limit(limit).all()
```

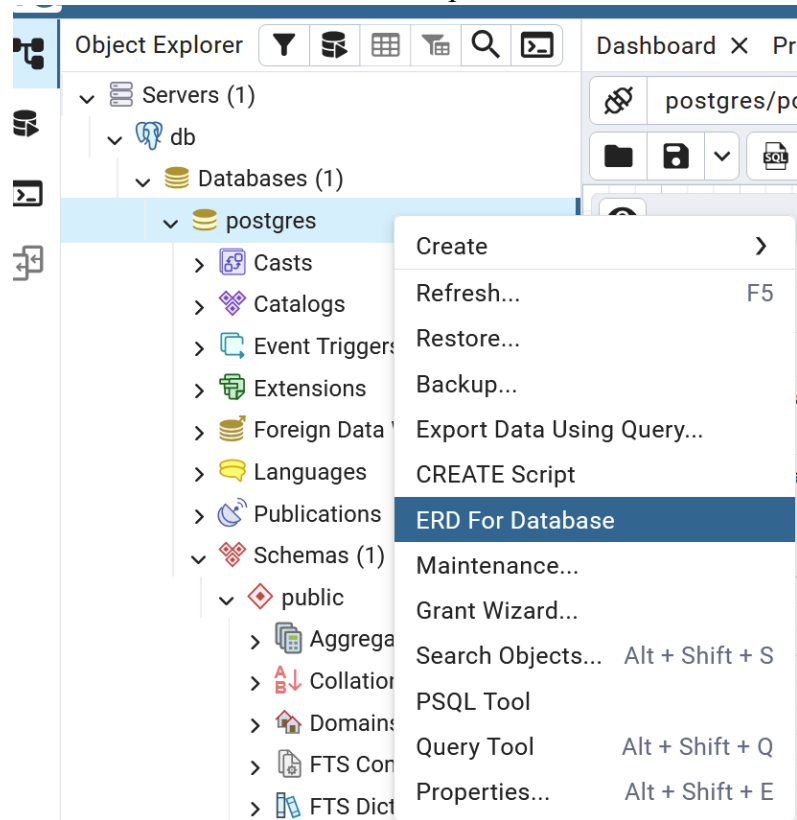
Теперь напишем функцию, модернизирующую уже существующую запись в БД. Это будет функция смены статуса у поста:

```
def update_post_status(session: Session, post_id: int, published: bool) -> bool:
    """Меняет статус публикации поста, возвращает True если успешно"""
    post = session.query(Post).filter(Post.id == post_id).first()
    if post is None:
        return False # Пост не найден

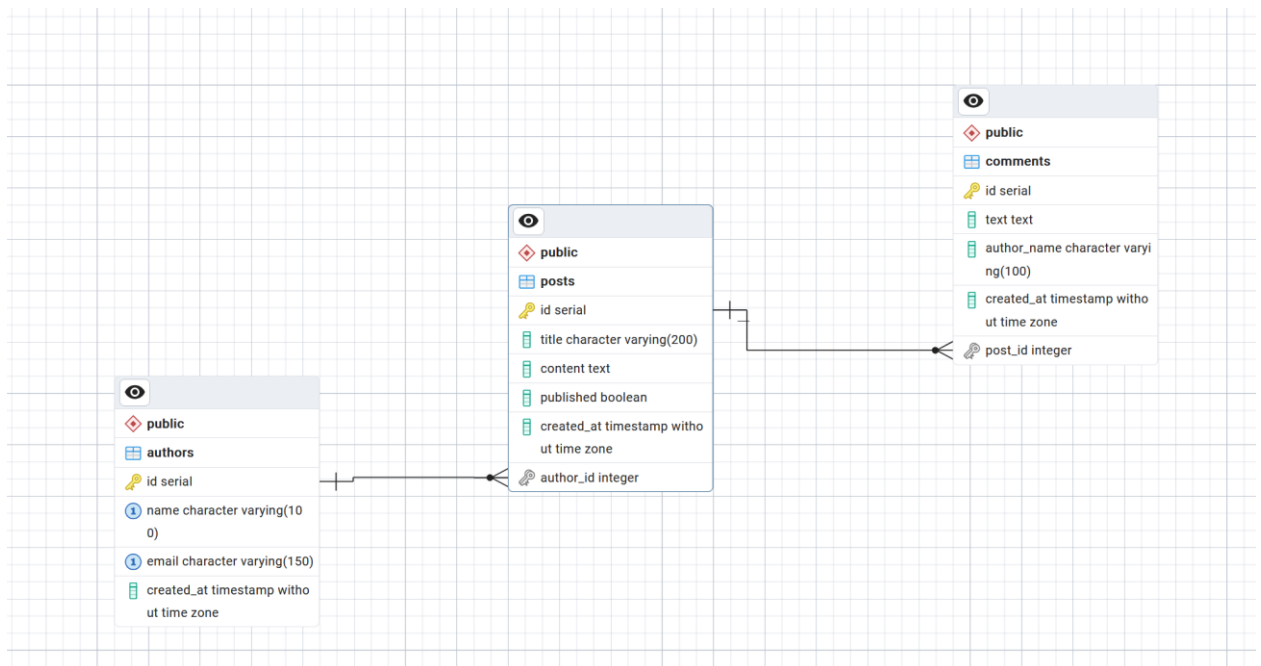
    post.published = published
    session.commit()
    return True
```

В данном случае мы ищем пост по его id, проверяем, есть ли он в БД, и, если есть, мы меняем его статус.

Теперь создадим более сложные запросы, для которых будут применяться созданные нами связи между таблицами в БД. Для начала, давайте посмотрим, какие связи создались в нашей БД при помощи pgAdmin. Для этого переходим в pgAdmin и на БД на панели слева нажимаем правой кнопкой мыши и в меню выбираем ERD for Database.



ERD – entity-relationship diagram или диаграмма сущность – связь, которая показывает связи между таблицами в БД. Схема связей между созданными нами таблицами выглядит следующим образом:



Далее создадим функцию добавления комментария к посту.

```

def add_comment(session: Session, post_id: int, author_name: str, text: str) -> Comment:
    """Добавляет комментарий к посту"""
    new_comment = Comment(
        post_id=post_id,
        author_name=author_name,
        text=text
    )
    session.add(new_comment)
    session.commit()
    session.refresh(new_comment)
    return new_comment
  
```

Теперь напишем функцию с более сложной логикой. Создадим аналитическую функцию, которая будет возвращать топ авторов постов по количеству созданных ими постов. Функция будет выглядеть следующим образом:

```

def get_top_authors_by_posts(session: Session, limit: int = 3) -> list[tuple[str, int]]:
    """
    Возвращает топ авторов по количеству постов.
    Пример возврата: [('Анна', 12), ('Иван', 8), ('Мария', 5)]
    """
    from sqlalchemy import func, desc

    # Запрос с агрегацией: считаем количество постов для каждого автора
    result = (session.query(
        Author.name,
        func.count(Post.id).label('post_count')
    )
    )
  
```

```
.join(Post) # Присоединяем таблицу постов по foreign key
.group_by(Author.id) # Группируем по автору
.order_by(desc('post_count')) # Сортируем по убыванию количества
.limit(limit) # Берём только топ-N
.all()

# result – это список кортежей: [('Анна', 12), ('Иван', 8), ...]
return result
```

Разберем основные моменты этой функции.

### **from sqlalchemy import func, desc**

func позволяет вызывать нативные SQL-функции (COUNT, SUM, MAX и т.д.). desc создаёт выражение ORDER BY ... DESC. Desc – функция сортировки по убыванию (asc – по возрастанию).

```
result = (session.query(
    Author.name,
    func.count(Post.id).label('post_count')
))
```

Указывает, что в SELECT попадёт имя автора и количество постов. .label('post\_count') даёт агрегированному столбцу псевдоним, чтобы на него можно было ссылаться в сортировке.

### **.join(Post)**

Метод .join(Post) в SQLAlchemy отвечает за связывание двух таблиц в одном SQL-запросе. Это означает: *"Возьми таблицу авторов и приклей к ней таблицу постов, совмещая строки по связи автор ↔ пост."*

### **.group\_by(Author.id)**

Преобразуется в GROUP BY authors.id, то есть выполняет группировку по заданному полю. Группировка (GROUP BY) – это механизм SQL, который "схлопывает" множество строк с одинаковыми значениями в определённых столбцах в одну логическую группу, после чего к каждой группе можно применить математическую операцию (посчитать, сложить, найти среднее и т.д.).

Обязательно при использовании COUNT. Группировка по id надёжнее, чем по name, так как корректно обрабатывает тёзок.

```
.order_by(desc('post_count'))
```

Генерирует ORDER BY post\_count DESC, то есть сортировка по убыванию по подсчитанному количеству постов у каждого автора. Использует псевдоним, заданный через .label().

Теперь проверим написанные нами функции. Для этого создадим файл main.py.

## **4. Создание точки входа в программу (файл main.py)**

В папке проекта создайте файл main.py и выполните в нем следующие импорты:



```
from database import SessionLocal, engine, Base
from models import Author, Post, Comment
from crud import *
```

Как видите, мы импортируем все созданные ранее нами модули.

Теперь напишем функцию main()

```
def main():
    # Создаём таблицы, если их нет (безопасно: если есть — ничего не меняет)
    Base.metadata.create_all(bind=engine)

    # Создаём сессию для работы с БД
    session = SessionLocal()

    try:
        print("Начинаем тестирование...\n")

        # 1. Создаём тестовых авторов
        print("Создаём авторов...")
        author1 = create_author(session, "Анна Петрова", "anna@example.com")
        author2 = create_author(session, "Иван Сидоров", "ivan@example.com")
        print(f"{author1.name} (id={author1.id})")
        print(f"{author2.name} (id={author2.id})\n")

        # 2. Создаём посты для Анны
        print("Создаём посты...")
        post1 = create_post(session, "Первый пост", "Это содержание первого поста. Оно достаточно длинное.", author1.id, published=True)
        post2 = create_post(session, "Черновик", "Этот пост пока не опубликован.", author1.id, published=False)
        post3 = create_post(session, "Пост Ивана", "Текст от Ивана.", author2.id, published=True)
        print(f"'{post1.title}' (опубликован)")
        print(f"'{post2.title}' (черновик)")
        print(f"'{post3.title}' (опубликован)\n")

        # 3. Добавляем комментарии к первому посту
        print("Добавляем комментарии...")
        add_comment(session, post1.id, "Читатель1", "Отличная статья, очень полезно!")
        add_comment(session, post1.id, "Читатель2", "Спасибо за материал, жду продолжения.")
        add_comment(session, post1.id, "Аноним", "Коротко.") # Этот комментарий тоже учтётся
        print("3 комментария добавлены к первому посту\n")

        # 4. Публикуем черновик
        print("Публикуем черновик...")
        success = update_post_status(session, post2.id, published=True)
        if success:
            print(f"'{post2.title}' теперь опубликован\n")
```

```

# 5. Выводим все опубликованные посты
print("📖 Все опубликованные посты:")
published = get_published_posts(session)
for post in published:
    print(f"'{post.title}' - автор: {post.author.name}")
print()

# 6. Топ авторов по количеству постов
print("👤 Топ авторов по количеству постов:")
top_authors = get_top_authors_by_posts(session, limit=3)
for rank, (name, count) in enumerate(top_authors, 1):
    print(f"{rank}. {name}: {count} пост(ов)")
print()

# 7. Проверка: поиск автора по email
print("Поиск автора по email...")
found = get_author_by_email(session, "anna@example.com")
if found:
    print(f"Найдено: {found.name}")
else:
    print("Автор не найден")

except Exception as e:
    print(f"Ошибка: {e}")
    session.rollback() # Отменяем все изменения при ошибке
finally:
    session.close() # Обязательно закрываем сессию!
print("\nТестирование завершено. Сессия закрыта.")

```

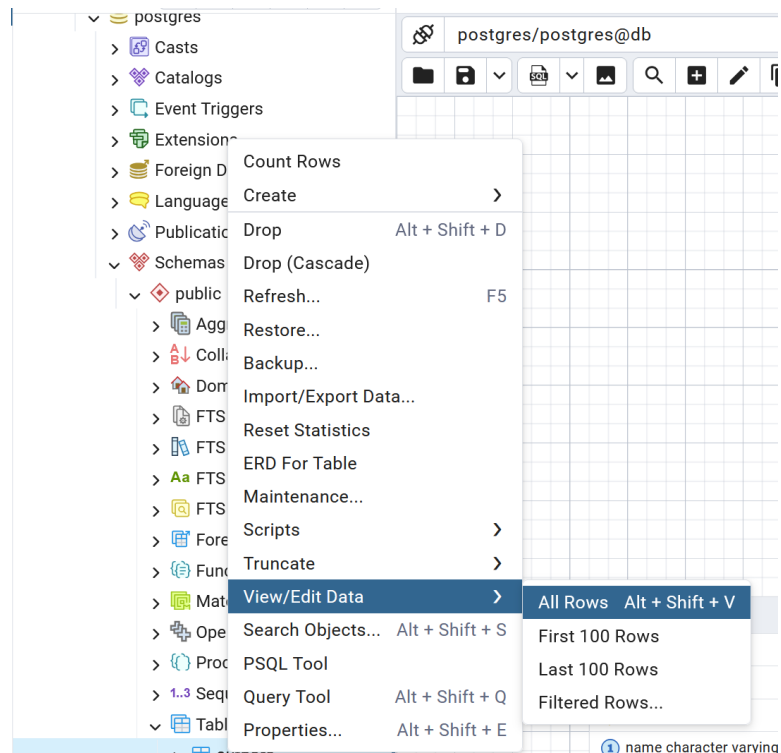
Далее добавляем классический синтаксис:

```

if __name__ == "__main__":
    main()

```

После выполнения кода в консоль будут выведены логи выполнения запросов. Если ошибок нет, то в БД создались новые записи и их можно посмотреть через pgAdmin. Для этого заходим в pgAdmin, найти интересующую таблицу (например, authors) и нажимаем View / Edit data All rows.



Если все отработало без ошибок, то вы увидите следующую таблицу:

|   | <b>id</b><br>[PK] integer | <b>name</b><br>character varying (100) | <b>email</b><br>character varying (150) | <b>created_at</b><br>timestamp without time zone |
|---|---------------------------|----------------------------------------|-----------------------------------------|--------------------------------------------------|
| 1 | 1                         | Анна Петрова                           | anna@example.com                        | 2026-05-05 09:56:13.013594                       |
| 2 | 2                         | Иван Сидоров                           | ivan@example.com                        | 2026-05-05 09:56:13.039965                       |

Аналогично можно просмотреть оставшиеся 2 таблицы.

### Самостоятельная работа

1. Написать функцию, которая находит автора по имени.
2. Написать функцию, которая выводит опубликованные посты за определенную дату.
3. Написать функцию добавления сразу нескольких авторов в БД
4. Написать функцию, которая находит пост по id и выводит сам пост и комментарии к нему (`get_post_with_comments`)
5. В `main.py` добавить тестирование новых функций.